

GloVe Word Embeddings Implemented from Scratch

Sparse co-occurrence modelling, hand-derived gradients and manual AdaGrad on WikiText-2, GPU-accelerated with PyTorch tensors

1. Introduction

Word embeddings map discrete vocabulary items into a continuous vector space in which geometric relations encode distributional similarity: words used in similar contexts end up close together, and consistent semantic contrasts (singular/plural, country/capital) appear as roughly parallel offsets. GloVe (Global Vectors, Pennington et al., 2014) learns such a space by factorising a global word-context co-occurrence matrix instead of scanning local windows stochastically as word2vec does. Because the corpus statistics are aggregated once up front, training operates on a compact list of non-zero counts and every gradient step uses global information.

The objective of this project is a genuine from-scratch implementation: the co-occurrence construction, the weighted least-squares objective, the analytic gradients and the AdaGrad optimiser are all written explicitly. PyTorch is used purely as a vectorised tensor and CUDA library - no autograd, no torch.optim, and no pretrained vectors enter training. Official GloVe vectors are downloaded only after training, for comparison.

2. Dataset and Preprocessing

WikiText-2 (raw) is downloaded programmatically from the Hugging Face mirror of the Salesforce/wikitext dataset and cached locally; the corpus itself is never vendored. Text is lowercased and tokenised with a regular expression that keeps alphabetic words (including internal apostrophes) and numbers while discarding standalone punctuation. A regex tokeniser is preferable here to a full NLP pipeline: WikiText is already lightly normalised, the tokeniser needs no model download (keeping the run reproducible), and dropping punctuation prevents commas from consuming context slots at the default window size of 1. Tokens occurring fewer than 5 times are removed outright rather than replaced by an <UNK> symbol, and the 20,000 most frequent survivors form the vocabulary.

Raw tokens	Unique raw types	Vocabulary	Retained tokens	Removed rare types	Tokens retained
1,760,383	63,985	20,000	1,683,936	43,985	95.66%

3. Co-occurrence Matrix

A symmetric context window is swept over the index stream. For a target at position p and a context at position q inside the window the entry $X[i,j]$ is incremented by $1/|p-q|$, so adjacent words contribute 1.0, words two apart 0.5, three apart 0.333, and so on; both directions are accumulated, making X symmetric. Out-of-vocabulary tokens are dropped before windowing, matching the reference GloVe implementation.

A dense $V \times V$ float32 matrix is never materialised. The counts are stored as (row, column, value) triplets, which is what makes the model trainable inside 4 GB of VRAM at all.

Vocabulary	Possible dense entries	Non-zero entries	Sparsity	Dense memory	Sparse memory	Compression
20,000	400,000,000	1,109,952	99.7225%	1.490 GB	12.70 MB	120.1x

4. GloVe Model, Objective and Optimiser

Each vocabulary entry owns a word vector w_i , a context vector $w_{\sim j}$ and two scalar biases b_i , $b_{\sim j}$, all randomly initialised. Training minimises the weighted least-squares objective

$$J = \sum_{ij} f(X_{ij}) * (w_i \cdot w_{\sim j} + b_i + b_{\sim j} - \log X_{ij})^2$$

with the weighting function

$$f(x) = (x / x_{\max})^{\alpha} \text{ if } x < x_{\max}, \text{ else } 1$$

which damps the influence of very frequent pairs without discarding them and prevents rare, noisy pairs from being over-weighted. Writing e_{ij} for the bracketed error and f_{ij} for the weight, the gradients derived by hand and implemented directly are

$$\begin{aligned} dJ/dw_i &= 2 f_{ij} e_{ij} w_{\sim j} & dJ/dw_{\sim j} &= 2 f_{ij} e_{ij} w_i \\ dJ/db_i &= 2 f_{ij} e_{ij} & dJ/db_{\sim j} &= 2 f_{ij} e_{ij} \end{aligned}$$

These are verified against autograd in the test suite. Parameters are updated with a hand-written AdaGrad step, accumulators initialised to 1.0 as in the reference implementation:

$$\text{acc} += g^2; \text{param} -= \text{lr} * g / \sqrt{\text{acc} + \text{eps}}$$

Within a mini-batch the per-sample gradients of a repeated row (a frequent word such as "the" occurs in many samples of the same batch) are summed into a single per-row gradient before the update is applied. Applying one row step per occurrence was observed to diverge to NaN in float32; summing first is both numerically stable and the correct mini-batch semantics. Final

embeddings are the sum $W + W_{\text{context}}$, as prescribed by the paper.

5. Training Setup and Results

The full triplet list stays in CPU memory and only the current mini-batch is transferred to the GPU; the persistent device tensors are the parameters, the AdaGrad accumulators and the gradient buffers. A CUDA out-of-memory error halves the batch size, empties the cache and retries, down to a configurable floor.

Setting	Value	Setting	Value
Embedding dimension	100	Epochs	30
Window size	1	Batch size	4,096
Learning rate	0.050	x_max / alpha	100.0 / 0.75
Device	cuda	GPU	NVIDIA GeForce RTX 3050 Laptop GPU
Training time (s)	24.24	Peak GPU memory (MB)	72.4700
Final training loss	0.011541	Training pairs	1,109,952

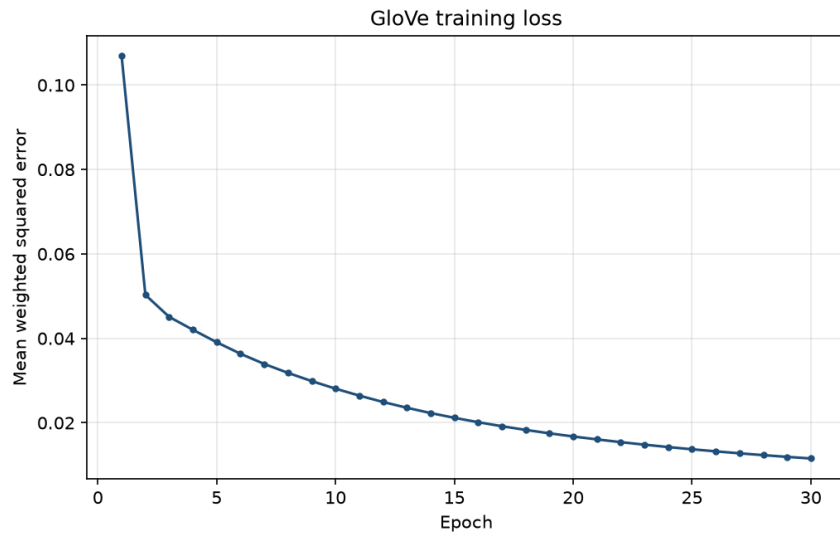


Figure 1: mean weighted squared error per epoch.

6. Evaluation

Query	Top-5 nearest neighbours (cosine)
king	james (0.88), henry (0.87), john (0.85), wood (0.84), constantine (0.84)
computer	science (0.83), fiction (0.83), stories (0.76), raffles (0.74), household (0.73)
city	bradford (0.88), kansas (0.88), norwich (0.86), york (0.83), town (0.82)
government	federal (0.96), canadian (0.85), isles (0.85), fountain (0.85), festival (0.85)
music	videos (0.92), video (0.91), sheet (0.87), awards (0.86), folk (0.80)

Analogies are solved as $a - b + c$ with the three source words excluded from the candidate set; quadruples containing an out-of-vocabulary word are marked OOV and excluded from the accuracy denominator. Of 16 analogies, 16 were evaluable and 3 were answered exactly, giving an accuracy of 18.8%. Coverage of the evaluation vocabulary was 56/56 words, an OOV rate of 0.0%.

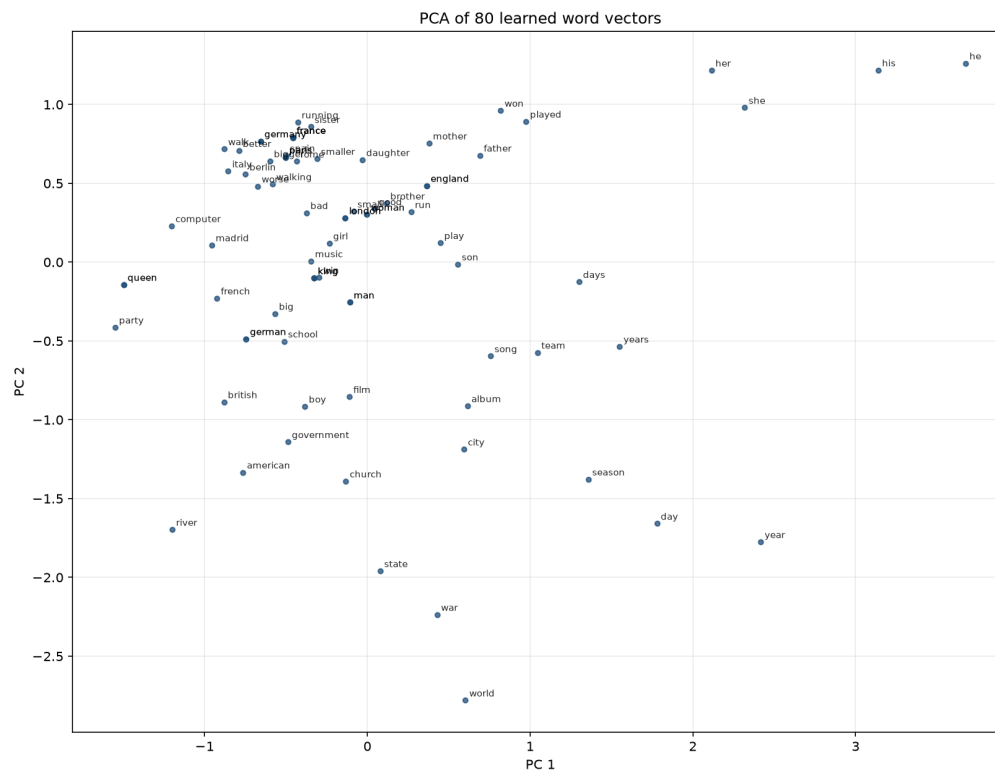


Figure 2: PCA projection of frequent learned word vectors.

7. Comparison with Official GloVe (glove.6B.100d)

Model	Vocabulary	Analogies evaluated	Correct	Accuracy	OOV rate
custom-wikitext2	20,000	16	3	18.8%	0.0%
glove.6B.100d	400,001	16	13	81.2%	0.0%

The official vectors are expected to win, and the reasons are structural rather than algorithmic. They are trained on roughly 6 billion tokens of Wikipedia plus Gigaword against the ~2 million tokens of WikiText-2 - three orders of magnitude more evidence per word - with a 400,000-word vocabulary, a window of 10 rather than 1, far longer optimisation, and tuned hyperparameters. Larger and more diverse data mostly helps the mid- and low-frequency words, which are exactly the ones an analogy set stresses; a window of 1 additionally captures little beyond immediate collocation, so our vectors encode syntagmatic adjacency more than broad topical similarity.

8. Ablation Study

Parameter	Value	Vocabulary	Non-zero pairs	Final loss	Analogy acc.	OOV %	Time (s)
embedding_dim	50	20,000	1,109,952	0.0160	18.8%	0.0%	17.1
embedding_dim	100	20,000	1,109,952	0.0115	18.8%	0.0%	24.3
embedding_dim	200	20,000	1,109,952	0.0086	12.5%	0.0%	39.6
window_size	1	20,000	1,109,952	0.0115	18.8%	0.0%	24.9
window_size	2	20,000	2,189,235	0.0120	18.8%	0.0%	51.2
window_size	5	20,000	4,901,453	0.0117	31.2%	0.0%	118.7
min_frequency	1	20,000	1,109,952	0.0115	18.8%	0.0%	25.5
min_frequency	5	20,000	1,109,952	0.0115	18.8%	0.0%	25.2
min_frequency	10	13,111	998,516	0.0129	18.8%	0.0%	20.0
epochs	10	20,000	1,109,952	0.0281	12.5%	0.0%	8.5
epochs	20	20,000	1,109,952	0.0168	18.8%	0.0%	17.0
epochs	30	20,000	1,109,952	0.0115	18.8%	0.0%	25.2

For embedding_dim, the lowest final loss (0.0086) came from embedding_dim=200, while the best analogy accuracy (18.8%) came from embedding_dim=50. For epochs, the lowest final loss (0.0115) came from epochs=30, while the best analogy accuracy (18.8%) came from epochs=20. For min_frequency, the lowest final loss (0.0115) came from min_frequency=5, while the best analogy accuracy (18.8%) came from min_frequency=1. For window_size, the lowest final loss (0.0115) came from window_size=1, while the best analogy accuracy (31.2%) came from window_size=5. Across the whole grid the strongest analogy accuracy was 31.2% at window_size=5. Note that final loss and analogy accuracy are not interchangeable: raising the embedding dimension or the epoch count lowers the training objective almost by construction, which says nothing on its own about how well the geometry generalises.

8b. CPU vs GPU Benchmark

Device	Epochs	Total time (s)	Mean epoch (s)	Final loss	Speedup
cpu	3	12.67	4.204	0.045084	4.58x
cuda	3	2.77	0.905	0.045084	4.58x

9. Limitations

The corpus is the dominant limitation: WikiText-2 supplies about three orders of magnitude fewer tokens than GloVe-6B, so co-occurrence counts for anything but the most frequent words are thin and their vectors correspondingly noisy. The 20,000-word cap and the frequency-5 cut-off remove a long tail of proper nouns and morphological variants, which shows up directly as out-of-vocabulary analogy items; nothing in the model can represent a word it never saw. The default window of 1 buys speed and sparsity at the cost of the wider topical context that helps semantic analogies. The analogy set used here is small and hand-assembled, so accuracy figures carry wide confidence intervals and should be read as directional. Being Wikipedia-derived, the corpus carries encyclopaedic register and its own demographic and topical biases into the geometry. Finally, results are sensitive to the learning rate, x_max and initialisation scale, only a slice of which the ablations explore, and the 4 GB VRAM budget bounds how far dimension and window could be pushed.

10. Conclusion

A complete GloVe pipeline was implemented from first principles and trained end to end: distance-weighted sparse co-occurrence construction, the weighted least-squares objective, hand-derived gradients validated against autograd, and a manual AdaGrad optimiser driven by mini-batches streamed to the GPU. On 20,000 vocabulary entries and 1,109,952 non-zero pairs the model

reached a final loss of 0.011541 in 24.2 s on NVIDIA GeForce RTX 3050 Laptop GPU, using 72.4700 MB of VRAM - comfortably inside a 4 GB budget. The exercise makes concrete why the sparse representation is not optional, how the weighting function stabilises learning across four orders of magnitude of co-occurrence counts, and how much of embedding quality is bought with corpus size rather than algorithmic cleverness.